CONTROL STRUCTURES

Definition

The C++ programming language has several structures for looping and conditional branching. Loops can execute a block of code as long as a specified condition is reached. Loops are handy because they save time, reduce errors, and they make code more readable.

Control structures represent the forms by which statements in a program are executed. Flow of control refers to the order in which the individual statements, instructions or function calls of a program are executed or evaluated.

IMPORTANCE OF CONTROL STRUCTURES

Generally, a program should execute the statements one by one until the defined end. This type of a program structure is called sequential structure. The functionality of this type of program is limited since it flows in a single direction. However, all high-level programming languages enable the programmer to change the flow of program execution. This is done by the use of control structures whose main benefits are to enable decision making and repetition as well as giving the power to do far more complex processing and provide flexibility with logic. The sophisticated logic is necessary for a program to solve complex problems.

The kinds of control flow statements supported by different languages vary, but can be categorized by

their effect:

- ✓ continuation at a different statement i.e. unconditional jump e.g. GoTo statements
- ✓ executing a set of statements only if some condition is met i.e. choice
- ✓ executing a set of statements zero or more times, until some condition is met i.e. loop
- ✓ executing a set of distant statements, after which the flow of control usually returns e.g.

subroutines/functions

TYPES OF CONTROL STRUCTURES

1. Sequence structures

Program statements are executed in the sequence in which they appear in the program.

2. Selection structures/Decision/Conditional Structures

Statement block is executed only if some condition is met. These structures include if, if/else, and switch. Selection structures are extensively used in programming because they allow the program to

decide an action based upon user's input or other processes for instance in password checking.

3. Repetition/Iterative structures

This is where a group of statements in a program may have to be executed repeatedly until some condition is satisfied. These include while, do/while and for

C++ has the following conditional statements:

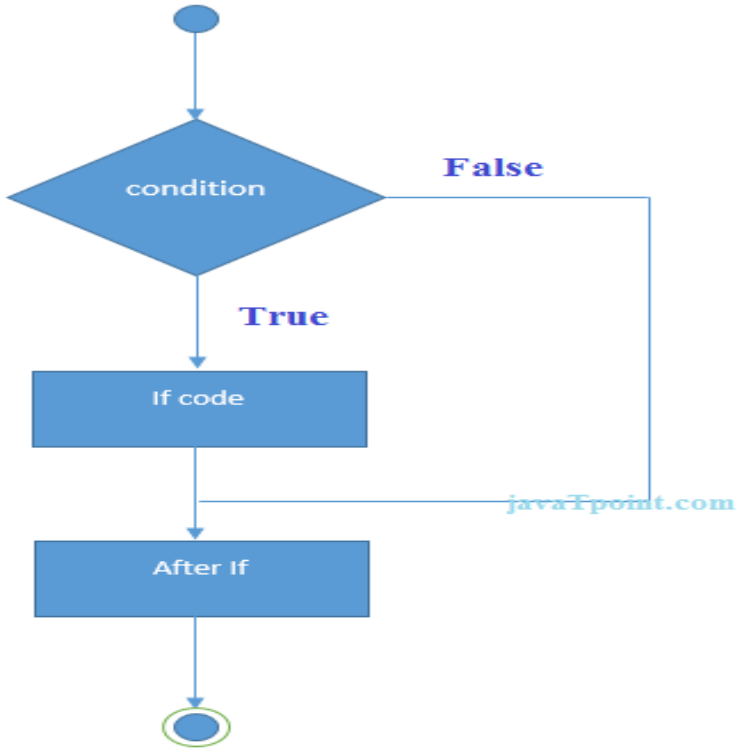
- Use if to specify a block of code to be executed, if a specified condition is true
- Use else to specify a block of code to be executed, if the same condition is false
- Use else if to specify a new condition to test, if the first condition is false
- Use switch to specify many alternative blocks of code to be executed

The if Statement

Use the `if` statement to specify a block of C++ code to be executed if a condition is `true`.

Syntax

```
if (condition) {
  // block of code to be executed if the condition is true
}
```



Example

```
1. #include <iostream>

2. using namespace std;

3.

4. int main () {

5.    int num = 10;

6.            if (num % 2 == 0)

7.            {

8.                cout<<"It is even number";

9.            }
```

```
10.         return 0;

11.      }
```

Output:

It is even number

The else Statement

Use the else statement to specify a block of code to be executed if the condition is false.

Syntax

```
if (condition) {
  // block of code to be executed if the condition is true
} else {
  // block of code to be executed if the condition is false
}
```

Example

```
int time = 20;
if (time < 18) {
  cout << "Good day.";
} else {
  cout << "Good evening.";
}
// Outputs "Good evening."
```

IF-else Statement

The C++ if-else statement also tests the condition. It executes if block if condition is true otherwise else block is executed.

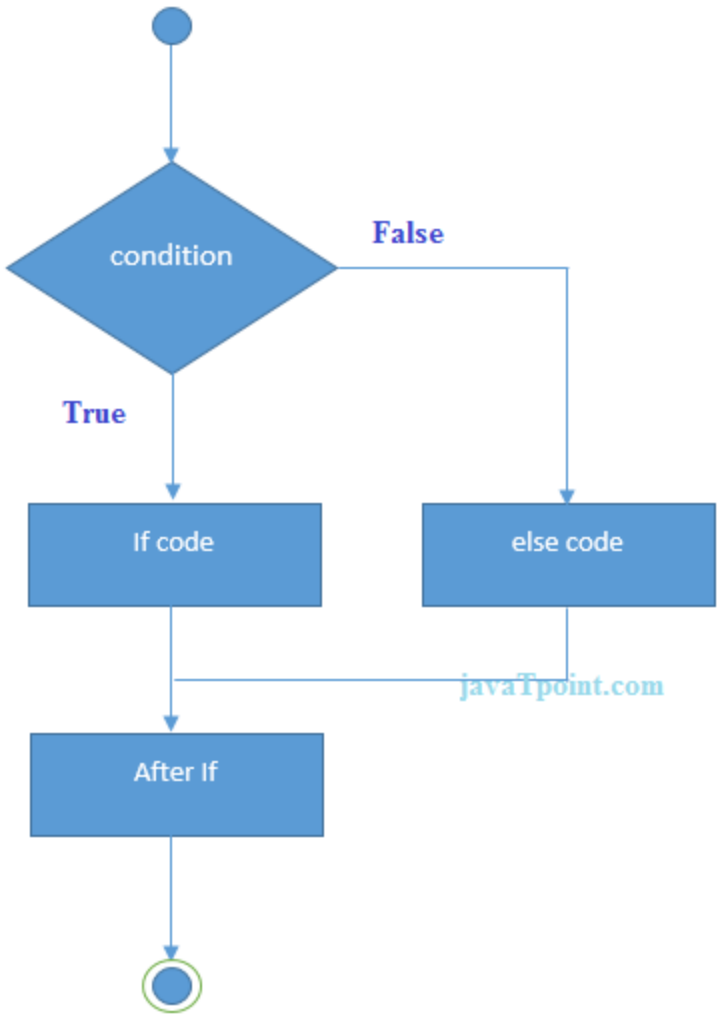
```
1. if(condition){

2. //code if condition is true

3. }else{

4. //code if condition is false

5. }
```



Example

```
1. #include <iostream>
```

```
using namespace std;
int main () {
   int num = 11;
            if (num % 2 == 0)
            {
                cout<<"It is even number";
            }
            else
                {
                    cout<<"It is odd number";
                }
        return 0;
     }
```

Output:

It is odd number

Example: with input from user

```
#include <iostream>
using namespace std;
int main () {
```

```
    int num;
    cout<<"Enter a Number: ";
    cin>>num;
            if (num % 2 == 0)
            {
                cout<<"It is even number"<<endl;
                }
                else
                {
                    cout<<"It is odd number"<<endl;
                }
        return 0;
    }
```

Output:

Enter a number:11

It is odd number

Output:

Enter a number:12

It is even number

IF-else-if ladder Statement

The C++ if-else-if ladder statement executes one condition from multiple statements.

```
1. if(condition1){

2. //code to be executed if condition1 is true

3. }else if(condition2){

4. //code to be executed if condition2 is true

5. }

6. else if(condition3){

7. //code to be executed if condition3 is true

8. }

9. ...

10.     else{

11.     //code to be executed if all the conditions are false

12.     }
```

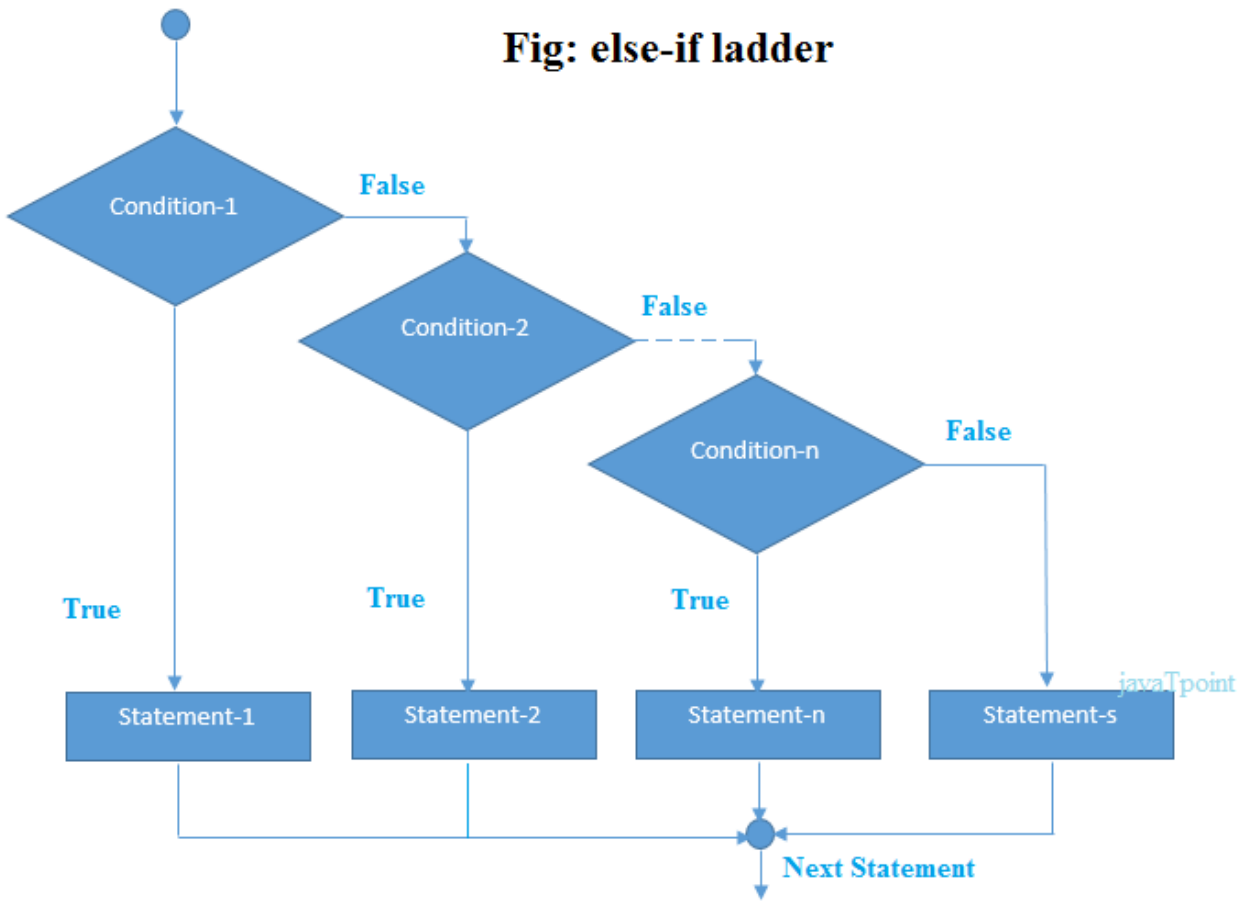


Fig: else-if ladder

Example

```
1. #include <iostream>

2. using namespace std;

3. int main () {

4.         int num;

5.         cout<<"Enter a number to check grade:";

6.         cin>>num;

7.              if (num <0 || num >100)
```

```
8.              {
9.                  cout<<"wrong number";
10.                 }
11.                 else if(num >= 0 && num < 50){
12.                     cout<<"Fail";
13.                 }
14.                 else if (num >= 50 && num < 60)
15.                 {
16.                     cout<<"D Grade";
17.                 }
18.                 else if (num >= 60 && num < 70)
19.                 {
20.                     cout<<"C Grade";
21.                 }
22.                 else if (num >= 70 && num < 80)
23.                 {
24.                     cout<<"B Grade";
25.                 }
26.                 else if (num >= 80 && num < 90)
27.                 {
28.                     cout<<"A Grade";
```

```
29.                 }

30.                 else if (num >= 90 && num <= 100)

31.                 {

32.                     cout<<"A+ Grade";

33.                 }

34.         }
```

Switch Statements

Use the switch statement to select one of many code blocks to be executed.

Syntax

```
switch(expression) {
  case x:
    // code block
    break;
  case y:
    // code block
    break;
  default:
    // code block
}
```

This is how it works:

- The switch expression is evaluated once
- The value of the expression is compared with the values of each case
- If there is a match, the associated block of code is executed
- The break and default keywords are optional, and will be described later in this chapter

The example below uses the weekday number to calculate the weekday name:

Example


```
int day = 4;
switch (day) {
  case 1:
    cout << "Monday";
    break;
  case 2:
    cout << "Tuesday";
    break;
  case 3:
    cout << "Wednesday";
    break;
  case 4:
    cout << "Thursday";
    break;
  case 5:
    cout << "Friday";
    break;
  case 6:
    cout << "Saturday";
    break;
  case 7:
    cout << "Sunday";
    break;
}
// Outputs "Thursday" (day 4)
```

The break Keyword

When C++ reaches a break keyword, it breaks out of the switch block.

This will stop the execution of more code and case testing inside the block.

When a match is found, and the job is done, it's time for a break. There is no need for more testing.

A break can save a lot of execution time because it "ignores" the execution of all the rest of the code in the switch block.

The default Keyword

The default keyword specifies some code to run if there is no case match:

Example

```
int day = 4;
switch (day) {
  case 6:
    cout << "Today is Saturday";
    break;
  case 7:
    cout << "Today is Sunday";
    break;
  default:
    cout << "Looking forward to the Weekend";
}
// Outputs "Looking forward to the Weekend"
```

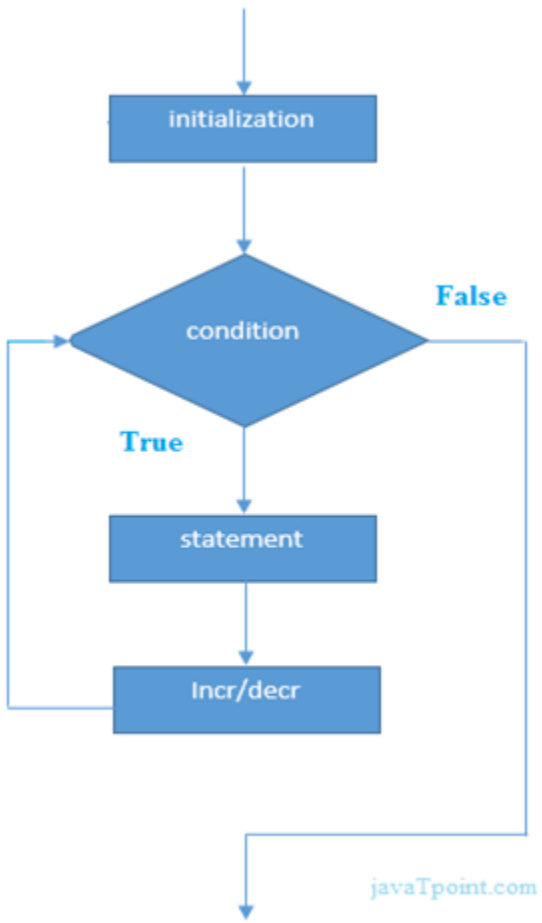
For Loop

The C++ for loop is used to iterate a part of the program several times. If the number of iteration is fixed, it is recommended to use for loop than while or do-while loops.

The C++ for loop is same as C/C#. We can initialize variable, check condition and increment/decrement value.

1. **for**(initialization; condition; incr/decr){
2. //code to be executed
3. }

Flowchart:



Example

```
1. #include <iostream>
2. using namespace std;
3. int main() {
4.         for(int i=1;i<=10;i++){
5.           cout<<i <<"\n";
6.         }
7.     }
```

Output:

1

2

3

4

5

6

7

8

9

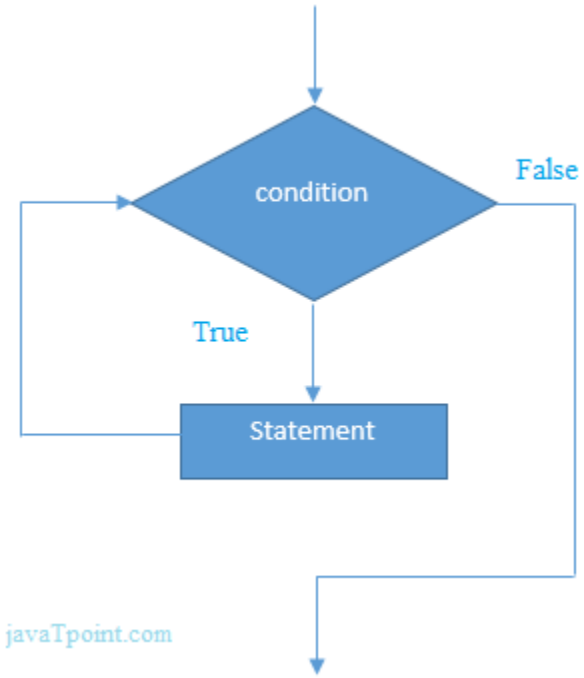
10

While loop

In C++, while loop is used to iterate a part of the program several times. If the number of iterations is not fixed, it is recommended to use while loop than for loop.

```
while(condition){
//code to be executed
}
```

Flowchart:



Example

```
1.  #include <iostream>
2.  using namespace std;
3.  int main() {
4.   int i=1;
5.        while(i<=10)
6.       {
7.          cout<<i <<"\n";
8.          i++;
9.        }
10.  }
```

Output:

1

2

3

4

5

6

7

8

9

10

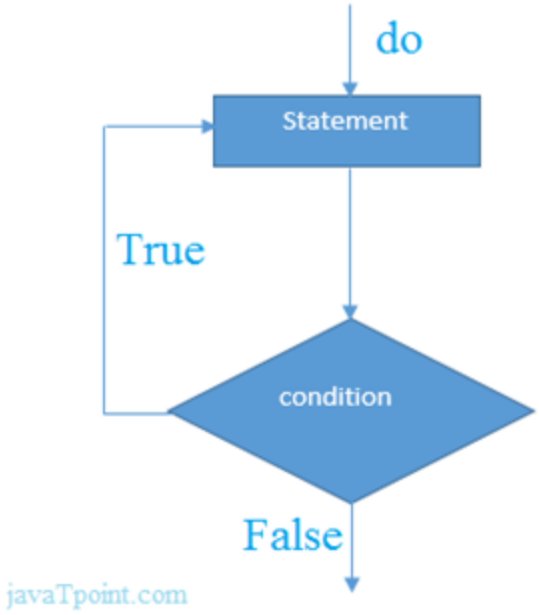
Do-While Loop

The C++ do-while loop is used to iterate a part of the program several times. If the number of iteration is not fixed and you must have to execute the loop at least once, it is recommended to use do-while loop.

The C++ do-while loop is executed at least once because condition is checked after loop body.

1. **do**{
2. //code to be executed
3. }**while**(condition);

Flowchart:



C++ do-while Loop Example

```
1. #include <iostream>
2. using namespace std;
3. int main() {
4.     int i = 1;
5.         do{
6.             cout<<i<<"\n";
7.             i++;
8.         } while (i <= 10) ;
9. }
```

Output:

1

2

3

4

5

6

7

8

9

10